

impactx 中 β 高 β ビームライン

コマンド早見

```
$ invoke clean
$ invoke track2impactx
$ invoke run
$ invoke post --refp --stat
$ eom png/refp/*.png
```

ディレクトリ構成

```
.
├── dat
│   ├── beamline_impactx.json ( track2impactx で生成、ビームラインのパラメータが入っている )
│   ├── beamline_track.json ( track2impactx で生成、sclinac.dat から拾ったパラメータが入る )
│   ├── parameters.json ( パラメータを記載するファイル：入力 )
│   ├── rfphase.csv ( track2impactx で生成、RFcavityとかで使用する位相情報などが書き出される )
│   └── visualize.json ( 可視化設定を記載するファイル：入力 )
├── impactx
│   └── main_impactx.py ( mainの実行ファイル：頻繁に変更しそうなパラメータはparameters.jsonから入力 )
├── lib
│   └── nk_toolkit ( python用のlibrary main_impactx.pyを簡素に書くためのルーチンや、変換コード、可視化コードを含む )
├── setvars.sh ( 1-ディレクトリ実行するための設定読み込みスクリプト パス・環境変数の設定 )
├── png ( 結果の画像 )
├── pyt ( 解析に利用する pythonコード置き場、初期は空 )
├── tasks.py ( 起動用ファイル invoke コマンドで使用する )
├── track
│   ├── sclinac.dat ( track ファイルの置き場, track2impactx で変換されるsclinac.dat はココに置く。 )
│   └── track.dat
```

実行方法

パス設定

- 画面を閉じない限り、動作の前に一度だけ実行で良い。
- pythonで使用している nk_toolkit (可視化ツール) について、PYTHONPATHにパスを設定するだけ。
- 固定パス (e.g.) ~/.python/lib/nk_toolkit) にnk_toolkitをおいて、~/.zshrc などに `export PYTHONPATH=HOME/.python/lib : PYTHONPATH` として、省略可能。

```
$ source lib/setvars.sh
```

準備 (track -> impactx 変換)

入力ファイル

- dat/parameters.json
- track/sclinac.dat (変換元trackのラティス構成)

出力ファイル

- dat/beamline_track.json
- dat/beamline_impactx.json
- dat/rfphase.csv

コマンド

```
$ invoke track2impactx
```

| トラッキング

| 入力ファイル

- dat/parameters.json
- dat/beamline_impactx.json

| コマンド

```
$ invoke run
```

| 出力ファイル

- impactx/diags/ 以下

(既にある場合, diags.old.xxxx に自動的にrenameされる.)

| ポスト処理

| 入力ファイル

- impactx/diags
- dat/visualize.json

| コマンド

```
$ invoke post --refp --stat --post --poincare --trajectory
```

| 出力ファイル

- png/refp/*
- png/stat/*
- png/ (その他)

| 基本プロットの対象

- refp : 基準粒子軌道
- stat : 統計(モーメント量)
- post : 追加の計算量 (lib/nk_toolkit/analyze_toolkit.py 中の get__postprocessed にて計算.)
- poincare : 位相空間プロット
- trajectory : 粒子軌道プロット

```
$ invoke post --refp --stat      ( refp & stat のみ )
```

```
$ invoke post --all              ( すべて出力 )
```

```
$ invoke post                    ( 出力なし )
```

ただし, tasks.py 内の

```
@invoke.task
def post( ctx, \
    refp=False, stat=False, post=False, poincare=False, \
    trajectory=False, all=False, ext=None, pcnffile="dat/visualize.json" ):
    """Run post-analysis script."""
```

に関して, 以下のように変更すれば デフォルト (無指定時) のフラグが解析とできる.

```
@invoke.task
def post( ctx, \
    refp=True, stat=True, post=False, poincare=False, \
    trajectory=False, all=False, ext=None, pcnffile="dat/visualize.json" ):
    """Run post-analysis script."""
```

デフォルトがTrueで, 単回で出力しないときは,

```
$ invoke post --no-stat
```

でstatなしになる。

parameters.json の設定

- パラメータが記載されたjson (json5) ファイル。
- 通常のjsonではなく、json5 (ルーズなjson)である点に注意
 - いわゆるケツカンマ問題をゆるす
 - 代わり、jq コマンド等が使用不可
 - pythonも import json5

シミュレーション設定

変数	内容
beam.charge.qe	粒子の電荷符号. 電子は -1, 陽子・重陽子は +1.
beam.charge.C	バンチ電荷 [C]. 電流I[A]とビーム周波数fb[Hz] より $Q=I/fb$.
beam.nparticles	マクロ粒子数.
beam.u.nucleon	粒子 1 個あたりの核子数. 重陽子は 2.0
beam.mass.amu	粒子質量 [amu]. 重陽子では 2.014
beam.Ek.MeV/u	核子あたりの運動エネルギー [MeV/u].
beam.freq.Hz	ビーム周波数 fb [Hz].
beam.harmonics	ハーモニック数. RF周波数は ($f_{RF} = h \times fb$)
beam.twiss.alpha	初期 Twiss (α) (ax, ay, az)
beam.twiss.beta	初期 Twiss (β). (bx, by, bz)
beam.emittance.geom	RMS 幾何エミッタンス [mm mrad]. Track-v38はfull, normalize
sim.max_level	AMR の最大レベル.
sim.n_cell	Poisson 方程式用メッシュ数.
sim.blocking_factor	AMR 計算で用いるブロッキングファクタ.
sim.nUse.elements	使用する要素数. null の場合は全要素を使用.

ImpactX の計算モード

変数	内容
mode.linear	線形計算を行うかどうか. true で線形要素を使用.
mode.space_charge	空間電荷計算の設定. false, "2D", "3D" を指定する.

track2impactx 時のパラメータ

変数	内容
translate.quad.skip	四重極磁石を使用しない場合 true (obsolete)
translate.drift.skip	ドリフトを使用しない場合は true (obsolete)
translate.cavity.skip	RF空洞を使用しない場合は true (obsolete)
translate.cavity.type	RF空洞の要素 "rfcavity" or "shortrf"
translate.cavity.rfgap	Trace3D 的な RFgap を使うか (現状は行列マップ)
translate.cavity.length	rfcavity の有限長 [m]
translate.cavity.phase	RF 空洞の位相 [deg].
translate.cavity.phase.fromfile	位相をファイルから読んで位相調整, 通常なし(null)
translate.cavity.options	RF空洞の詳細追加設定.
translate.quad.factor	四重極磁石強度に掛ける係数 (一律で弱める)(obsolete)
translate.quad.options	四重極磁石の単位系, スライス数, アパーチャなどを指定
translate.drift.options	ドリフトの詳細追加設定. スライス数, アパーチャ等を指定

- options は、impactx readthedocsに記載のパラメータ指定を指定する。(nsliceやアパーチャ、電場のFourier係数、等)
- 一部、使用していないけど残している (obsolete) モード等あり。

visualize.json の設定

構成

- json (json5)

```
visualize.json
├─ files
├─ refp
│  └─ settings ( ルーチン内で使う変数 )
│  └─ default ( refpで共通のプロットオプション )
│  └─ s-xp ( s-vs-xp_ref のプロット )
│      └─ config ( s-xp のプロットオプション )
│      └─ plots ( プロットする変数, files記載の変数 ただしrefpは物理量がrefがつく. 複数指定可能. )
│      └─ option ( プロットルーチンに渡す追加命令. )
├─ stat
│  └─ 同様...
├─ poincare
│  └─ 同様...
├─ post
│  └─ 同様...
├─ trajectory
│  └─ 同様...
```

プロットオプションとして設定可能な内容

- lib/nk_toolkit/plot/config.json (共通プロットコンフィグファイル)、及び、
- lib/nk_toolkit/gplot1d.py (1Dプロットルーチン) を参照
- 上記のconfig.jsonにはデフォルト値を記載。
- visualize.json (各プロット個別設定) > visualize.json (プロット種別(refp等)設定) > visualize.json (全体設定) > config.json の順で反映。

設定例

- "ax1.x.range" : { "auto": true, "min": 0.0, "max": 1.0, "num": 11 },
 - auto : true のときは、自動レンジ (min, max は自動取得, numは目盛りの数)
 - auto : false のときは、min, max は手動指定、numは 目盛りの数

インストール

- HP参照 (<https://impactx.readthedocs.io/en/latest/install/users.html>)
- pip が簡単、だけど、並列できない (noacc)
- cmake は煩雑、だけど、並列できる
 - まずは、pip インストールで試し、cmakeで本番環境とするのがよいと思います。

impactx以外のコード部分で使用するpython ライブラリ

- pipが必要なライブラリは適宜、pipにて取得ください。(ライブラリが見つからない、と出た場合。)
- 下記は参考例

```
$ python -m pip install numpy scipy matplotlib pandas h5py tqdm invoke json5
```

もし、必要な他ライブラリがあれば、pip インストールで基本的には入手できるはず。

- もしも、nk_toolkitが見つからない、と出た場合だけ、それは環境 (パス) の問題。
 - setvars.sh で設定したパスが入っているかを確認 (export | grep PYTHONPATHなどで表示。)

impactx の pip インストール (impactx本体)

```
$ python -m pip install impactx-noacc
```

cmake

- 環境ごとに、何が足りていないのか、様子を見ながら足りないソフトをインストールする必要あり。

- 基本は、HPの情報とし、terminalの返答に応じて、適宜、修正してください。
- 下記は、西田が必要だった修正を記載 (使用するライブラリのソースコンパイル+intel コンパイラの使用、など。)

前提環境

本記事では、以下の環境で ImpactX をソースからビルドする。

- WSL
- Ubuntu 24.04
- Intel oneAPI を使用 (intel c++ でないgcc等の場合、環境変数設定 (setvars.sh) 等は不要)
- Python バインディングを有効化
- ADIOS2, openPMD-api, ImpactX をすべてソースからビルド
- ソースコードは \$HOME/opt/ 以下に配置 (作業ディレクトリ)
- インストール先は /opt/ 以下 (配置ディレクトリ sudo が必要な場合、注意)

注意事項

- WSLのUbuntu 24.04 では、 apt 経由で導入した ADIOS2 を用いるとリンク時に問題が発生。
- ADIOS2 および openPMD-api をソースからビルドした。
 - (実際は、hdf5などのコンパイルがまだの場合、必要かもしれない。)

ソースコードの取得

作業用ディレクトリを作成し、ADIOS2、openPMD-api、ImpactX のソースコードを取得する。

```
mkdir -p ~/opt
cd ~/opt

git clone https://github.com/ornladios/ADIOS2.git adios2
git clone https://github.com/openPMD/openPMD-api.git openpmd
git clone https://github.com/ECP-WarpX/impactx.git impactx
```

環境設定

Intel oneAPI の環境を有効化し、MPI コンパイラを指定する。

```
source /opt/intel/oneapi/setvars.sh

export CC=mpicc
export CXX=mpicxx
```

ADIOS2 のビルド

ADIOS2 を MPI および Python サポート付きでビルドする。

```
cd ~/opt/adios2
mkdir build && cd build
cmake .. -DADIOS2_BUILD_EXAMPLES=ON -DCMAKE_INSTALL_PREFIX=/opt/adios2 -DADIOS2_USE_Python=ON -DADIOS2_USE_MPI=ON -
DADIOS2_C_COMPILER=mpicc -DADIOS2_CXX_COMPILER=mpicxx > cmake_build.log 2>&1
make -j8
sudo su
source /opt/intel/oneapi/setvars.sh
make install
exit
```

openPMD のビルド

openPMD- を MPI、Python、HDF5、ADIOS2 サポート付きでビルドする。

HDF5 が /opt/hdf5/hdf5-1.10.11_oneAPI にインストール済みであることを前提。(HDF5がない場合、コンパイル必要かも。)

```
cd ~/opt/openPMD
mkdir build && cd build
cmake .. -DopenPMD_USE_MPI=ON -DopenPMD_USE_PYTHON=ON -DopenPMD_USE_HDF5=ON -DopenPMD_USE_ADIOS2=ON -
DHDF5_ROOT=/opt/hdf5/hdf5-1.10.11_oneAPI -DCMAKE_INSTALL_PREFIX=/opt/openPMD -
DADIOS2_DIR=/opt/adios2/lib/cmake/adios2 -DCMAKE_EXE_LINKER_FLAGS="-ladios2_mpi_cxx11" > cmake_build.log 2>&1
make -j8
sudo cmake --build . --target install
```

ImpactX のビルド

ImpactX を Python バインディング、FFT、openPMD対応付きでビルドする。

```
cd ~/opt/impactx
cmake -S . -B build -DImpactX_PYTHON=ON -DCMAKE_INSTALL_PREFIX=/opt/impactx -DImpactX_FFT=ON -
DCMAKE_PREFIX_PATH="/opt/openPMD;/opt/adios2" -DImpactX_openpmd_internal=OFF
cmake --build build -j 8
sudo su
source /opt/intel/oneapi/setvars.sh
cmake --build build --target install
sudo ln -s /opt/impactx/bin/impactx.MPI.OMP.DP.OMPMD /usr/local/bin/impactx
```

CMake オプションに関するメモ

ImpactX のビルド時には、openPMD-api の場所を直接指定する OPENPMD_DIR のようなオプションは見当たらなかった。

そのため、CMAKE_PREFIX_PATH に openPMD-api と ADIOS2 のインストール先を指定する。

```
-DCMAKE_PREFIX_PATH="/opt/openPMD;/opt/adios2"
```

また、ImpactX はデフォルトでは openPMD-api を内部で取得してビルドする設定になっている。その挙動を避け、外部にインストール済みの openPMD-api を使うために、次のオプションを指定する。

```
-DImpactX_openpmd_internal=OFF
```

PATH などの設定

.zshrc などに以下を追記する。

```
export LD_LIBRARY_PATH=$LD_LIBRARY_PATH:/opt/impactx/lib
export LD_LIBRARY_PATH=$LD_LIBRARY_PATH:/opt/openPMD/lib
export PYTHONPATH=$PYTHONPATH:/opt/openPMD/lib/python3.12/site-packages
```